4

4.1

StarryOS

moss Linux

`fork` COW I/O I/O

4.2

4.2.1 AddrSpace

`ProcessData` `Arc<Mutex<AddrSpace>>`

`fork` `AddrSpace` `CLONE_VM` clone

`AddrSpace`

```
pub struct AddrSpace {
    // 允  用    程使用的完整虚  地址范
    va_range: VirtAddrRange,
    // 按起始地址    的虚  内存区域
    areas: MemorySet<Backend>,
    // 当前已  生效的虚    到物理  映射
    pt: PageTable,
}
```

4-1

`va_range` `areas` `pt`

`map` `unmap` `protect` `populate_area` `handle_page_fault`

4.2.2 MemorySet MemoryArea

`MemorySet` `MemoryArea` `MemoryArea` `Backend`

```rust
pub struct MemorySet<B: MappingBackend> {
    // 以区域起始地址    ，便于  找相  区域和    重
    areas: BTreeMap<B::Addr, MemoryArea<B>>,
}

pub struct MemoryArea<B: MappingBackend> {
    // 一段    的虚  地址范
    va_range: AddrRange<B::Addr>,
    // 整段区域共享的  、写、  行等  限
    flags: B::Flags,
    // 决定区域中的  面从何  取得
    backend: B,
}
```

4-2 MemorySet MemoryArea

4.2.3 PageTable

4 KiB

CPU

Backend “

” MemoryArea

MemoryArea

axhal PageTable

```rust
// axhal 根据    目      具体  表  型
#[cfg(target_arch = "riscv64")]
pub type PageTable = Sv39PageTable<PagingHandlerImpl>;

#[cfg(target_arch = "loongarch64")]
pub type PageTable = LA64PageTable<PagingHandlerImpl>;

// 多体系      表共享的核心
pub struct PageTable64<M, PTE, H> {
    root_paddr: PhysAddr,
    _phantom: PhantomData<(M, PTE, H)>,
}
```

4-3

4.2.4 Backend

Backend AddrSpace

COW Backend

```rust
#[derive(Clone)]
#[enum_dispatch(BackendOps)]
pub enum Backend {
    // 按固定偏移建立虚  地址到物理地址的映射
    Linear(LinearBackend),
    // 管理匿名  和私有映射，并在 fork 后  行写    制
    Cow(CowBackend),
    //
    Shared(SharedBackend),
    // 将文件  存  映射到用  地址空
    File(FileBackend),
}
```

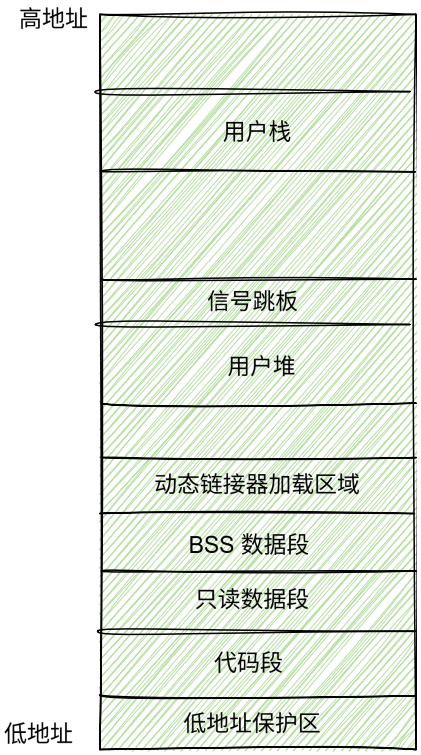
4-4 Backend

Backend `populate` `clone_map`

File Backend

4.3

4-1 RISC-V64 LoongArch64 ELF



4-1 moss

4.3.1

`new_user_aspace_empty`

ELF

`rt_sigreturn` moss

4.3.2 ELF

ELF “ ”

`PT_LOAD`

ELF

```
// ph 是一个 ELF 程序段描述。
// PT_LOAD 表示  段必  装入用  地址空  。
for ph in program_headers
    .iter()
    .filter(|ph| ph.get_type() == Ok(Type::Load))
{
    // virtual_addr 是段在程序中的虚  地址，
    // elf_base 是主程序或      接器的加  基址。
    let vaddr = ph.virtual_addr as usize + elf_base;
    let seg_start = VirtAddr::from_usize(vaddr);

    // 段起点可能没有按      ，seg_pad 保存  内偏移。
    // mem_size 是  段装入内存后的  大小。
    let seg_pad = vaddr.align_offset_4k();
    let seg_align_size =
        align_up_4k(ph.mem_size as usize + seg_pad);

    let backend = Backend::new_cow(
        seg_start,
        PageSize::Size4K,
        // cache 保存 ELF 文件的数据  。
        FileBackend::Cached(cache.clone()),
        // offset 是段内容在 ELF 文件中的起始位置。
        ph.offset,
        // file_size 是文件中    存在的数据大小；
        // mem_size 多出的部分不  取文件，保持  零。
        Some(ph.offset + ph.file_size),
    );

    uspace.map(
        seg_start.align_down_4k(),
```

```
        seg_align_size,
        // 根据段  志  置  、写、  行  限。
        mapping_flags(ph.flags),
        false, // 只登  区域，不立即装入物理
        backend,
    )?;
}
```

4-5 ELF

Backend ELF

BSS

moss ELF `fork/exec`

BusyBox

shebang musl glibc

4.3.3

moss `USER_HEAP_BASE`

64 KiB `brk`

Cow Backend

`fork`

4.3.4

`mmap` moss

```
pub fn sys_mmap(
    addr: usize,   // 建  使用的虚  地址；固定映射  目  地址
    length: usize, // 映射区域的  度
    prot: u32,     //
    flags: u32,    // 私有、共享、匿名或固定映射等方式
    fd: i32,       // 文件映射使用的文件描述符
    offset: isize, // 映射内容在文件中的起始偏移
) -> AxResult<isize>
```

4-6 mmap

Backend `find_free_area`

	Backend	
`MAP_PRIVATE	MAP_ANONYMOUS`	`Cow`	
`MAP_PRIVATE`	`Cow`	
`MAP_SHARED	MAP_ANONYMOUS`	`Shared`	
`MAP_SHARED`	`File`	
	`Linear`	

4-1 mmap Backend

moss `MAP_SHARED_VALIDATE` `MAP_FIXED_NOREPLACE` `MAP_GROWSDOWN` `MAP_LOCKED`

offset `mremap`

`MREMAP_MAYMOVE` moss `mmap`

`/proc/[pid]/maps` `s` `p`

4.4

4.4.1

`AddrSpace::map` MemoryArea `populate`

`populate_area` ELF `mmap` `MAP_POPULATE`

`populate_area` MemoryArea Backend Backend

COW

`VirtMem`

4.4.2

Cow Backend

Shared Backend

Linear Backend

File Backend Page Cache Backend

Page Cache I/O

4.4.3

MemoryArea `populate_area`

`AddrSpace`

4.5

moss StarryOS

moss FILE_CACHES

GLOBAL_PAGE_CACHE

“ inode ” FILE_CACHES

CachedFileRecord GLOBAL_PAGE_CACHE LRU

4.5.1

```
const PAGE_SIZE: usize = 4096;
// 全局最多  存 16K 个普通文件  ，共 64 MiB
const GLOBAL_FILE_CACHE_PAGES: usize = 16 * 1024;

// 同一文件只建立一份  存
static FILE_CACHES:
    RwLock<BTreeMap<FileCacheKey, Arc<CachedFileRecord>>>;

// 所有普通文件的数据  共用一个全局  存；
// 内部的 LruCache                          LRU 算法  行  面淘汰。
static GLOBAL_PAGE_CACHE: Mutex<GlobalPageCache>;

struct GlobalPageKey {
    cache_id: u64, // 文件  存
    pn: u32,       // 文件内  号
}
```

4-7

CachedFile

256

File Backend 4

4.5.2

ELF CachedFile::get_or_create

libc mmap

I/O

libc

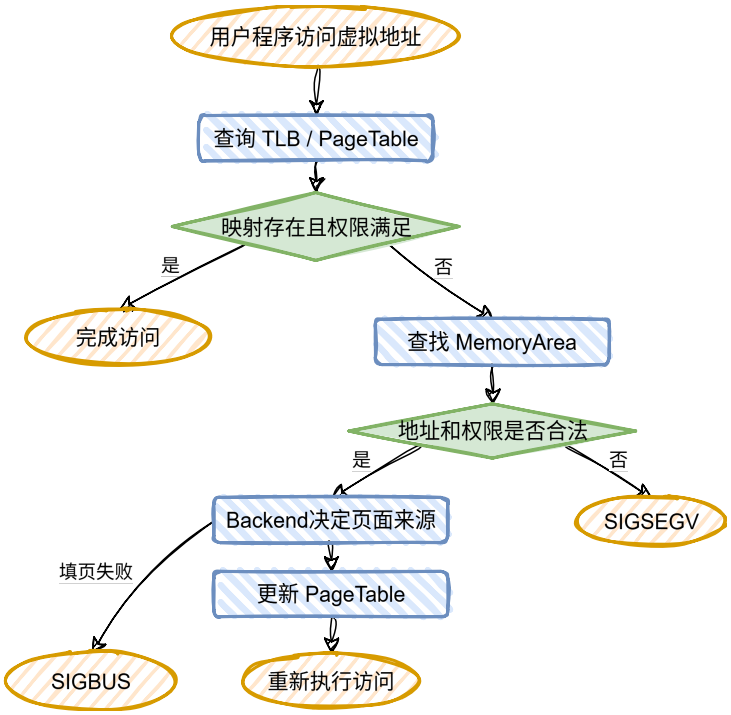
Cow Backend

File Backend

4.5.3

64 MiB LRU

16 64 KiB I/O `fsync`



4-2

Backend AddrSpace::handle_page_fault

Backend populate

```
// 先根据故障地址找到所属的 MemoryArea。
// 找不到区域  明  地址不属于  程的合法虚  空  。
if let Some(area) = self.areas.find(vaddr) {
    let flags = area.flags();

    // 本次    的  型必  被区域  限允  ，
    // 例如写    需要区域本身  有 WRITE
    if flags.contains(access_flags) {
        let page_size = area.backend().page_size();

        // Backend 以      位填充，因此先把故障地址向下    。
        return area.backend()
            .populate(
                VirtAddrRange::from_start_size(
                    vaddr.align_down(page_size),
                    page_size as usize,
                ),
                flags,
                access_flags,
                &mut self.pt.modify(),
            )
            .is_ok();
    }
}
```

```
// 地址不存在或  限不匹配  ，缺    理失  。
false
```

4-8

CPU

Backend

4.6.3

MemoryArea

`SIGSEGV`

`SIGBUS`

grow-down		
	`SIGSEGV`	
	`SIGBUS`	

4-2

moss `MAP_GROWSDOWN` `mremap` `SIGBUS`

LoongArch64 `SIGBUS`

Linux

4.7

4.7.1 fork

`fork` `AddrSpace::try_clone`

`fork`

`CLONE_VM` clone `AddrSpace`

4.7.2 COW

COW Cow

Backend

```
match frame_ref_count {
    1 => {
        // 已无其他地址空  共享，直接恢复写  限
        pt.protect(vaddr, flags)?;
    }
    _ => {
        // 仍被共享，复制数据并改映射到新  面
        let new_frame = alloc_new_frame(false)?;
        copy_page(paddr, new_frame);
        pt.remap(vaddr, new_frame, flags)?;
        drop_old_reference(paddr);
    }
}
```

4-9 COW

4.7.3 moss COW

StarryOS COW `fork` moss `usize`

moss Cow Backend `fork` `fork/exec`

4.8

4.8.1

`munmap` moss `AddrSpace` `AddrSpace::unmap` `MemorySet::unmap` `MemorySet` `MemoryArea`

```
pub fn unmap(
    &mut self,
    start: B::Addr,
    size: usize,
    page_table: &mut B::PageTable,
) -> MappingResult {
```

```
    let range = AddrRange::try_from_start_size(start, size)
        .ok_or(MappingError::InvalidParam)?;

    //                        MemoryArea。
    self.areas.retain(|_, area| {
        if area.va_range().contained_in(range) {
            area.unmap_area(page_table).unwrap();
            false
        } else {
            true
        }
    });

    // 左  界相交：收  区域尾部，或先切分再收  。
    // 右  界相交：收  区域  部。
    // 收  与切分  程中都会  用 backend.unmap 清除  表  。
    ...
}
```

4-10 MemorySet

`mprotect` `mremap` `mprotect`

`mremap` `unmap`

`brk`

`execve` `AddrSpace::clear`

`AddrSpace` `Drop` `clear`

exec Backend

4.9

StarryOS `AddrSpace` `MemoryArea` `PageTable` Backend

Backend CPU

`fork` Backend COW

moss `mmap` `mremap` grow-down

COW fork File Backend